Andrew Kozlevcar

Section 11:00

Question 1

In [1]:
```
import numpy as np
import pandas as pd
import plotly.express as px
import sklearn
from pathlib import Path
from urllib.request import urlretrieve
from zipfile import ZipFile
from io import BytesIO
from scipy.special import expit
from sklearn.base import clone
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.multiclass import OneVsRestClassifier
from sklearn.model_selection import GroupShuffleSplit
from sklearn.dummy import DummyClassifier
from sklearn.metrics import (
    accuracy_score, f1_score, log_loss,
    confusion_matrix, classification_report
)
from IPython.display import display
```

In [2]:
```
archive_path = Path("uci_har.zip")
data_url = (
    "https://archive.ics.uci.edu/static/public/240/"
    "human+activity+recognition+using+smartphones.zip"
)

if not archive_path.exists():
    try:
        urlretrieve(data_url, archive_path)
    except Exception as error:
        archive_path.unlink(missing_ok=True)
        raise RuntimeError(
            "Download failed. Upload the original UCI archive "
            "as uci_har.zip, then rerun this cell."
        ) from error

archive = ZipFile(archive_path)

# Some UCI downloads contain a second ZIP inside the first.
if not any(n.endswith("train/X_train.txt") for n in archive.namelist()):
```

```
    inner_name = next(n for n in archive.namelist() if n.endswith(".zip"))
    archive = ZipFile(BytesIO(archive.read(inner_name)))

required_files = [
    "features.txt", "activity_labels.txt",
    "train/X_train.txt", "train/y_train.txt", "train/subject_train.txt",
    "test/X_test.txt", "test/y_test.txt", "test/subject_test.txt"
]

tables = {}
for suffix in required_files:
    member = next(n for n in archive.namelist()
                  if n == suffix or n.endswith("/" + suffix))
    tables[suffix] = pd.read_csv(
        archive.open(member), sep=r"\s+", header=None
    )

print("Loaded the UCI modeling files.")
```

```
Loaded the UCI modeling files.
```

In [3]:

```
features = tables["features.txt"]
feature_names = features[0].astype(str) + "__" + features[1]

X_train = tables["train/X_train.txt"].copy()
X_test = tables["test/X_test.txt"].copy()
X_train.columns = feature_names
X_test.columns = feature_names

y_train = tables["train/y_train.txt"][0]
y_test = tables["test/y_test.txt"][0]
subject_train = tables["train/subject_train.txt"][0]
subject_test = tables["test/subject_test.txt"][0]

label_names = {
    1: "Walking", 2: "Walking upstairs", 3: "Walking downstairs",
    4: "Sitting", 5: "Standing", 6: "Lying down"
}
classes = np.array(sorted(label_names))
class_names = [label_names[k] for k in classes]

assert X_train.shape[1] == X_test.shape[1] == 561
assert X_train.columns.is_unique
assert not X_train.isna().any().any()
assert not X_test.isna().any().any()
assert set(subject_train).isdisjoint(set(subject_test))
assert set(y_train) == set(y_test) == set(classes)

display(pd.DataFrame({"Activity code": classes, "Activity": class_names}))
print("Original training shape:", X_train.shape)
print("Original test shape:", X_test.shape)
display(X_train.iloc[:5, :8])
```

	Activity code	Activity
0	1	Walking
1	2	Walking upstairs
2	3	Walking downstairs
3	4	Sitting
4	5	Standing
5	6	Lying down

```
Original training shape: (7352, 561)
Original test shape: (2947, 561)
```

	1__tBodyAcc-mean()-X	2__tBodyAcc-mean()-Y	3__tBodyAcc-mean()-Z	4__tBodyAcc-std()-X	5__tBodyAcc-std()-Y	6__tBody st
0	0.288585	-0.020294	-0.132905	-0.995279	-0.983111	-0.91
1	0.278419	-0.016411	-0.123520	-0.998245	-0.975300	-0.96
2	0.279653	-0.019467	-0.113462	-0.995380	-0.967187	-0.97
3	0.279174	-0.026201	-0.123283	-0.996091	-0.983403	-0.99
4	0.276629	-0.016570	-0.115362	-0.998139	-0.980817	-0.99

In [4]:
```
splitter = GroupShuffleSplit(n_splits=1, test_size=5, random_state=6021)
fit_idx, val_idx = next(splitter.split(X_train, y_train, groups=subject_trai

X_fit = X_train.iloc[fit_idx]
X_val = X_train.iloc[val_idx]
y_fit = y_train.iloc[fit_idx]
y_val = y_train.iloc[val_idx]
subject_fit = subject_train.iloc[fit_idx]
subject_val = subject_train.iloc[val_idx]

assert set(subject_fit).isdisjoint(set(subject_val))
assert set(subject_fit).isdisjoint(set(subject_test))
assert set(subject_val).isdisjoint(set(subject_test))
assert set(y_fit) == set(y_val) == set(classes)

split_summary = pd.DataFrame({
    "Set": ["Fitting", "Validation", "Final test"],
    "Windows": [len(X_fit), len(X_val), len(X_test)],
    "Participants": [subject_fit.nunique(), subject_val.nunique(),
                     subject_test.nunique()]
})
display(split_summary)
print("Predictors used by every model:", X_fit.shape[1])
print("Participant overlap between sets: none")
```

	Set	Windows	Participants
0	Fitting	5442	16
1	Validation	1910	5
2	Final test	2947	9

```
Predictors used by every model: 561
Participant overlap between sets: none
```

In [5]:
```
variability_feature = "4__tBodyAcc-std()-X"
gravity_feature = "41__tGravityAcc-mean()-X"

plot_data = X_fit[[variability_feature, gravity_feature]].copy()
plot_data["Activity"] = y_fit.map(label_names).to_numpy()

fig = px.scatter(
    plot_data, x=variability_feature, y=gravity_feature,
    color="Activity", opacity=0.3,
    labels={
        variability_feature: "Body acceleration variability (X axis)",
        gravity_feature: "Mean gravity acceleration (X axis)"
    },
    title="Two features: movement variability and posture information"
)
fig.show()

class_counts = y_fit.value_counts().sort_index()
class_counts.index = [label_names[k] for k in class_counts.index]
display(class_counts.rename("Fitting windows").to_frame())
```

	Fitting windows
Walking	924
Walking upstairs	801
Walking downstairs	737
Sitting	944
Standing	1009
Lying down	1027

Part A

As per the description, one row represents a 2.56-second window of activity from one of the 30 volunteers, and the row additionally contains 561 engineered predictors that describe this window of activity; the response variable is one of the activities (Walking, Walking upstairs, Walking downstairs, Sitting, Standing, Lying down). Activity codes 1-6 should not be modeled as a continous numerical outcome because it would force an arbitrary numeric spacing and ordering of the nominal outcomes; codes 1 and 6 are not

neccesarily further apart as codes 1 and 4, but this is what the model implicitly suggests. Under such a scheme, it is also possible to output predictions that are fractional, which does not have a clear interpretation; is it "between" the two classes, or is it somehow being counterbalanced by multiple classes further away? Classification models avoid this representation issue by treating each activity as a separate category and estimating a probability for each of them.

Part B

Walking upstairs (light blue) overlaps signficiantly with walking (magenta). Roughly speaking, mean gravity acceleration models orientation (upright or sideways) and body acceleration variability represents up and down movement. With respect to these two predictors (especially the first), these activities look very similar. In reality, our predictor space is 561 dimensions, and this visual is projecting the points on to a two dimensional surface. In this higher dimensional space, it is possible that there is definitive spacing of the groups, which would symbolize that a combination of predictors (including those not visualized here) can distinguish between the walking upstairs and walking effectively.

Part C

We know for certain that adjacent windows overlap by at most 50%, and we can plausibly assume the windows of a single person are strongly correlated. In either case, if the windows of a person appear in training and atleast one of validation/test, some of the information contained in this validation/test window has already been incorporated into the model. This constitutes data leakage and means that our estimates will be overly optimistic. The participant ID allows us to group all of the windows of a particular person for approrpiate splitting, and there are several reasons we can justify its ommision as a predictor. For one, it would have to be one hot encoded, and our model would only be able to predict inputs with predictor IDs that exist in our training data. Conceptually, incorporating participant IDs would also encourage the recognition of person-level patterns, which undermines our goal of identify universal patterns that align with a certain activity.

Question 2

In [6]:
```
pipe_ovr = Pipeline(steps=[
    ("preprocess", StandardScaler()),
    ("model", OneVsRestClassifier(LogisticRegression(
        l1_ratio=0, C=1.0, solver="lbfgs",
        max_iter=5000, random_state=123
    )))
])
```

```
pipe_multi = Pipeline(steps=[
    ("preprocess", StandardScaler()),
    ("model", LogisticRegression(
        l1_ratio=0, C=1.0, solver="lbfgs",
        max_iter=5000, random_state=123
    ))
])

pipe_ovr.fit(X_fit, y_fit)
pipe_multi.fit(X_fit, y_fit)

assert np.array_equal(pipe_ovr.classes_, classes)
assert np.array_equal(pipe_multi.classes_, classes)

print("Number of OvR binary models:",
      len(pipe_ovr.named_steps["model"].estimators_))
print("Multinomial coefficient matrix shape:",
      pipe_multi.named_steps["model"].coef_.shape)
```

```
Number of OvR binary models: 6
Multinomial coefficient matrix shape: (6, 561)
```

In [7]:

```
p_ovr = pipe_ovr.predict_proba(X_val)
p_multi = pipe_multi.predict_proba(X_val)
raw_ovr = expit(pipe_ovr.decision_function(X_val))

assert np.allclose(p_ovr, raw_ovr / raw_ovr.sum(axis=1, keepdims=True))
assert np.allclose(p_ovr.sum(axis=1), 1)
assert np.allclose(p_multi.sum(axis=1), 1)

i = 0
true_code = int(y_val.iloc[i])
true_col = np.flatnonzero(classes == true_code)[0]

probabilities = pd.DataFrame({
    "Activity": class_names,
    "OvR raw sigmoid": raw_ovr[i],
    "OvR normalized": p_ovr[i],
    "Multinomial softmax": p_multi[i]
})

print("Actual activity:", label_names[true_code])
display(probabilities.round(6))
print("Column sums:")
display(probabilities.drop(columns="Activity").sum().round(6))

window_rows = []
for name, p in [("OvR", p_ovr), ("Multinomial", p_multi)]:
    predicted_col = p[i].argmax()
    window_rows.append({
        "Model": name,
        "Predicted activity": class_names[predicted_col],
        "Largest probability": p[i, predicted_col],
        "True-class probability": p[i, true_col],
        "Single-window log loss": -np.log(max(p[i, true_col], 1e-15))
    })
```

```
window_results = pd.DataFrame(window_rows)
display(window_results.round(6))
```

```
Actual activity: Standing
```

	Activity	OvR raw sigmoid	OvR normalized	Multinomial softmax
0	Walking	0.000208	0.000208	0.000000
1	Walking upstairs	0.000000	0.000000	0.000000
2	Walking downstairs	0.000000	0.000000	0.000000
3	Sitting	0.000411	0.000411	0.000338
4	Standing	0.998919	0.999305	0.999659
5	Lying down	0.000075	0.000075	0.000003

```
Column sums:
OvR raw sigmoid        0.999614
OvR normalized         1.000000
Multinomial softmax    1.000000
dtype: float64
```

	Model	Predicted activity	Largest probability	True-class probability	Single-window log loss
0	OvR	Standing	0.999305	0.999305	0.000695
1	Multinomial	Standing	0.999659	0.999659	0.000341

Part A

OvR fits six binary classification models, where each activity has its own model. For each of these models, $y_i = 1$ if observation i's activity corresponds to the model's activity and 0 otherwise. We choose the coeffcients (which determine the linear score and probabilities) such that they minimize the loss function comprised of log loss (-$\sum_{i=1}^{n} y_i \log(p_i) + (1 - y_i) \log(1 - p_i)$) and the penalty term of $\sum_{j=1}^{p} \beta_j^2$.

Alternatively, multinomial fits a single model, which chooses a number of coefficients equal to the total number of coefficients across the six OvR models (if we assume no baseline and p fewer if we assume a baseline). To choose these coefficients, we minimize the loss function comprised of $\sum_{i=1}^{n} -\eta_{i,y_i} + \log(\sum_{j=1}^{k} e^{\eta_{i,j}})$ and the penalty term of $\sum_{u=1}^{p} \sum_{v=1}^{k} \beta_{u,v}^2$. The key difference between the two procedures is that OvR fits coefficients for classes separately whereas multinomial fits coefficients for classes jointly.

By the above discussion, the fact that the two models output probabilities that sum to one implies nothing about how they obtained these probabilities (and thus their loss

functions)--it just reflects that they both either normalized their probability outputs or employ a procedure that guarantees built-in normalization.

Part B

For the displayed validation window, the actual activity is standing. OvR predicts standing with largest probability 0.999305 whereas multinomial predicts standing with probability 0.999659. Assuming usual decision rule means choosing the class with the highest probability, a model could in fact predict an activity with just a predicted 0.38 probability. In settings where there are three or more classes, a probability need not be greater than 0.5 to be the largest probability. For instance, suppose for a three class model that the outputted probabilities are 0.31, 0.31, and 0.38. These sum up to 1, and 0.38 is the largest probability.

Part C

The single window log loss for OvR is 0.000695, which is larger than the single window log loss for multinomial, 0.000341. As demonstrated above (and verified by the log loss), multinomial assigns a better probability (as in closer to 1) for the observed outcome for this window. This comparison indicates that the multinomial better predicts (slightly) this single window, but it makes no claims about the relative prediction performance of the two models on the other windows. Judging overall performance would require comparing the average log loss across all windows.

Question 3

In [8]:
```
baseline = DummyClassifier(strategy="prior")
baseline.fit(X_fit, y_fit)
p_dummy = baseline.predict_proba(X_val)
y_dummy = baseline.predict(X_val)

baseline_results = pd.DataFrame([{
    "Model": "Class-frequency baseline",
    "Validation accuracy": accuracy_score(y_val, y_dummy),
    "Validation macro F1": f1_score(
        y_val, y_dummy, average="macro", zero_division=0
    ),
    "Validation log loss": log_loss(y_val, p_dummy, labels=classes)
}])

models = {"OvR L2 C=1": pipe_ovr, "Multinomial L2 C=1": pipe_multi}
model_rows = []
for name, model in models.items():
    p_fit = model.predict_proba(X_fit)
```

```
    p_val = model.predict_proba(X_val)
    pred_val = model.predict(X_val)
    model_rows.append({
        "Model": name,
        "Training log loss": log_loss(y_fit, p_fit, labels=classes),
        "Validation log loss": log_loss(y_val, p_val, labels=classes),
        "Validation accuracy": accuracy_score(y_val, pred_val),
        "Validation macro F1": f1_score(
            y_val, pred_val, average="macro", zero_division=0
        )
    })

results = pd.DataFrame(model_rows)
display(baseline_results.round(4))
display(results.round(4))
```

	Model	Validation accuracy	Validation macro F1	Validation log loss
0	Class-frequency baseline	0.199	0.0553	1.7813

	Model	Training log loss	Validation log loss	Validation accuracy	Validation macro F1
0	OvR L2 C=1	0.0203	0.1377	0.9555	0.9549
1	Multinomial L2 C=1	0.0132	0.1676	0.9419	0.9393

In [9]:
```
y_pred_multi = pipe_multi.predict(X_val)
cm = confusion_matrix(y_val, y_pred_multi, labels=classes)
cm_df = pd.DataFrame(cm, index=class_names, columns=class_names)
cm_df.index.name = "Actual activity"
cm_df.columns.name = "Predicted activity"
display(cm_df)

fig = px.imshow(
    cm_df, text_auto=True, aspect="auto",
    labels={"x": "Predicted activity", "y": "Actual activity",
            "color": "Windows"},
    title="Validation confusion matrix: multinomial, C=1"
)
fig.update_layout(height=550)
fig.show()

report = classification_report(
    y_val, y_pred_multi, labels=classes, target_names=class_names,
    output_dict=True, zero_division=0
)
report_df = pd.DataFrame(report).T.loc[
    class_names, ["precision", "recall", "f1-score", "support"]
]
display(report_df.round(4))
```

Predicted activity / Actual activity	Walking	Walking upstairs	Walking downstairs	Sitting	Standing	Lying down
Walking	295	0	7	0	0	0
Walking upstairs	7	264	0	1	0	0
Walking downstairs	14	24	211	0	0	0
Sitting	0	0	0	304	28	10
Standing	0	0	0	16	349	0
Lying down	0	0	0	0	4	376

	precision	recall	f1-score	support
Walking	0.9335	0.9768	0.9547	302.0
Walking upstairs	0.9167	0.9706	0.9429	272.0
Walking downstairs	0.9679	0.8474	0.9036	249.0
Sitting	0.9470	0.8889	0.9170	342.0
Standing	0.9160	0.9562	0.9357	365.0
Lying down	0.9741	0.9895	0.9817	380.0

Part A

For the baselin, the validation accuracy was 0.199 and the validation macro F1 was 0.0553. On the other hand, the validation accuracy for multinomial was 0.9419, and its validation macro F1 was 0.9393. Based on both metrics, multinomial appears to perform better.

It can be shown that accuracy is a weighted average of recall for each of the classes, and the weight is determined by the frequencey of a class in the validation windows. In this way, the accuracy metric can be dominated by performance on the most frequently occuring classes, obscuring performance in the less frequently occuring classes. Alternatively, Macro F1 takes the unweighted average of the F1 scores for each class; F1 score for a class is the harmonic mean of precision and recall. Since all classes are weighted equally by this metric, performance on less frequently occuring classes make an equal impression. F1 score also factors in precision, which accuracy does not directly do.

Part B

The largest off-diagonal cell is 28 windows; the actual activity is sitting, and the predicted acitvity is standing. Sitting recall is the proportion of true sitting windows that were predicted to be sitting windows. To obtain recall manually, we identify the sitting row; the row sum represents the total number of actual sitting windows. 304 of the actual sitting windows were predicted to be sitting, 28 were predicted to be standing, and 10 were predicted to be lying down, giving a total of $304 + 28 + 10 = 342$ actual sitting windows. The diagonal entry symbolizes the number of true sitting windows that were predicted to be sitting windows: 304. Thus, sitting recall is $\frac{304}{342} \approx 0.889$, which is confirmed by the table below. In context, this means that of the actual sitting windows, the model correctly predicted 88.9% of them; of the roughly 11% missed, approximately 74% of them were wrongly labeled as standing, suggesting that the model may struggle to differentiate between these two activities in some cases.

Part C

The model with the smallest validation log loss is OvR with 0.1377, followed by multinomial (0.1676) and the baseline (1.7813). The accuracy ranking agrees; OvR has the highest validation accuracy of (0.9555) followed by mutlinomial (0.9419) and the baseline (0.199). While the ranking of accuracy and log loss agreed in this case, this is not generally true. Conceptually, accuracy is determined solely by the dominant probability, but it does not consider information about just how dominant this probability was. For instance, accuracy cannot distinguish between a prediction that assigns the actual window's activity a dominant probability of 1 and a prediction that assigns the actual window's activity a dominant probability of 0.17, which, under the right circumstances in a six-class model, can happen. On the other hand, log loss accounts for the confidence of the prediction by making loss a function of the predicted class probability for the actual class and penalizing predictions whose actual class probability is further from 1. In this way, one model may have higher accuracy because it correctly assigns a dominant probability to the actual class for more of the validation windows, but each dominant probability is not a high probability relative to the other classes, and its incorrect predictions assign the actual class probability to a very small value. On the other hand, a model may correctly predict a smaller share of the validation windows, but its misses are "close" (actual class probability is still relatively high compared to the dominant probability), and its correct predictions are strong (assign near 1 probabilities to the actual class).

Training log loss should not determine the preferred model. In greater detail, a model captures true signal (persistent patterns that are a population-level characteristic) and sample noise (volatile patterns that are attributes of the sample). Ideally, we want to build a model that capture the most true signal and limits sample noise; this will allow predictions to generalize to unseen data better. To understand whether our model generally fit signal or sample artifacts (noise), we need to assess it on new data. We

should expect this new sample to have the same signal but different noise, so if the model's performance is relatively poor (as compared to other models) on this new data, then it suggests that it captured relatively more noise than signal when it originally fit the training data. If we only assess model performance on the training data, we implicitly attribute all of the observed fit to signal, which leads to an over optimistic assesment of our model's performance.

Question 4

In [10]:
```
ridge = Pipeline(steps=[
    ("preprocess", StandardScaler()),
    ("model", LogisticRegression(
        l1_ratio=0, C=0.01, solver="lbfgs",
        max_iter=5000, random_state=123
    ))
])

lasso = Pipeline(steps=[
    ("preprocess", StandardScaler()),
    ("model", LogisticRegression(
        l1_ratio=1, C=0.01, solver="saga",
        max_iter=5000, random_state=123
    ))
])

ridge.fit(X_fit, y_fit)
lasso.fit(X_fit, y_fit)
models["Multinomial L2 C=0.01"] = ridge
models["Multinomial L1 C=0.01"] = lasso

# Rebuild the comparison table using the same fitting/validation rows.
model_rows = []
for name, model in models.items():
    p_fit = model.predict_proba(X_fit)
    p_val = model.predict_proba(X_val)
    pred_val = model.predict(X_val)
    model_rows.append({
        "Model": name,
        "Training log loss": log_loss(y_fit, p_fit, labels=classes),
        "Validation log loss": log_loss(y_val, p_val, labels=classes),
        "Validation accuracy": accuracy_score(y_val, pred_val),
        "Validation macro F1": f1_score(
            y_val, pred_val, average="macro", zero_division=0
        )
    })

results = pd.DataFrame(model_rows)
display(results.round(4))
print("Lasso iterations:", lasso.named_steps["model"].n_iter_)
```

	Model	Training log loss	Validation log loss	Validation accuracy	Validation macro F1
0	OvR L2 C=1	0.0203	0.1377	0.9555	0.9549
1	Multinomial L2 C=1	0.0132	0.1676	0.9419	0.9393
2	Multinomial L2 C=0.01	0.0819	0.2236	0.9068	0.9053
3	Multinomial L1 C=0.01	0.2777	0.3783	0.9010	0.8934

```
Lasso iterations: [2296]
```

In [11]:
```
multinomial_models = {
    "Ridge C=1": pipe_multi,
    "Ridge C=0.01": ridge,
    "Lasso C=0.01": lasso
}

coefficient_rows = []
for name, model in multinomial_models.items():
    beta = model.named_steps["model"].coef_
    nonzero = np.abs(beta) > 1e-6
    active = nonzero.any(axis=0)
    coefficient_rows.append({
        "Model": name,
        "Coefficient L2 norm": np.linalg.norm(beta),
        "Nonzero class slopes": int(nonzero.sum()),
        "Active predictors": int(active.sum()),
        "All-six-zero predictors": int((~active).sum())
    })

coefficient_summary = pd.DataFrame(coefficient_rows)
display(coefficient_summary.round(4))

lasso_coefficients = pd.DataFrame(
    lasso.named_steps["model"].coef_.T,
    index=X_fit.columns, columns=class_names
)
print("Lasso coefficient table: rows = features; columns = classes")
display(lasso_coefficients.head(10).round(5))

fig = px.bar(
    coefficient_summary, x="Model", y="Active predictors",
    text="Active predictors",
    title="Predictors retained in at least one class score"
)
fig.update_yaxes(range=[0, X_fit.shape[1] * 1.1])
fig.show()
```

	Model	Coefficient L2 norm	Nonzero class slopes	Active predictors	All-six-zero predictors
0	Ridge C=1	10.1266	3366	561	0
1	Ridge C=0.01	2.4724	3366	561	0
2	Lasso C=0.01	3.7132	64	60	501

Lasso coefficient table: rows = features; columns = classes

	Walking	Walking upstairs	Walking downstairs	Sitting	Standing	Lying down
1__tBodyAcc-mean()-X	0.0	0.00000	0.0	0.0	0.0	0.0
2__tBodyAcc-mean()-Y	0.0	-0.03486	0.0	0.0	0.0	0.0
3__tBodyAcc-mean()-Z	0.0	0.00000	0.0	0.0	0.0	0.0
4__tBodyAcc-std()-X	0.0	0.00000	0.0	0.0	0.0	0.0
5__tBodyAcc-std()-Y	0.0	0.00000	0.0	0.0	0.0	0.0
6__tBodyAcc-std()-Z	0.0	0.00000	0.0	0.0	0.0	0.0
7__tBodyAcc-mad()-X	0.0	0.00000	0.0	0.0	0.0	0.0
8__tBodyAcc-mad()-Y	0.0	0.00000	0.0	0.0	0.0	0.0
9__tBodyAcc-mad()-Z	0.0	0.00000	0.0	0.0	0.0	0.0
10__tBodyAcc-max()-X	0.0	0.00000	0.0	0.0	0.0	0.0

In [16]:
```
results[["Model", "Validation log loss"]]
```

Out[16]:

	Model	Validation log loss
0	OvR L2 C=1	0.137679
1	Multinomial L2 C=1	0.167650
2	Multinomial L2 C=0.01	0.223592
3	Multinomial L1 C=0.01	0.378262

In [12]:
```
best_name = results.loc[results["Validation log loss"].idxmin(), "Model"]
print("Selected using validation log loss:", best_name)
```

```
display(results.loc[results["Model"] == best_name].round(4))

# Refit the selected configuration on all 21 original training people.
# clone removes fitted state; the scaler is also refitted here.
final_model = clone(models[best_name])
final_model.fit(X_train, y_train)
print("Final model fitted. Its configuration is now fixed.")
```

```
Selected using validation log loss: OvR L2 C=1
```

	Model	Training log loss	Validation log loss	Validation accuracy	Validation macro F1
0	OvR L2 C=1	0.0203	0.1377	0.9555	0.9549

```
Final model fitted. Its configuration is now fixed.
```

In [13]:
```
p_test = final_model.predict_proba(X_test)
y_pred_test = final_model.predict(X_test)

final_results = pd.DataFrame([{
    "Selected model": best_name,
    "Test accuracy": accuracy_score(y_test, y_pred_test),
    "Test macro F1": f1_score(
        y_test, y_pred_test, average="macro", zero_division=0
    ),
    "Test log loss": log_loss(y_test, p_test, labels=classes)
}])
display(final_results.round(4))
```

	Selected model	Test accuracy	Test macro F1	Test log loss
0	OvR L2 C=1	0.9579	0.9581	0.1545

Part A

By definition, C is inversely related to λ. Since increasing λ increases the influence of the penalty term and results in a stronger regularization, decreasing C results in a stronger regularization. So, $C = 0.01$ is applying a stronger regularization than $C = 1$; this is confirmed by the coefficient L2 norms, where $C = 0.01$ multinomial ridge results in an L2 norm of 2.4724 and the $C = 1$ multinomial ride results in an L2 norm of 10.1266. Based on validation log loss, we see that the $C = 1$ model results in a validation log loss of 0.167650, whereas the validation log loss of the $C = 0.01$ model is 0.223592. In this run, since the $C = 1$ model has lower log loss, the results suggest that stronger shrinkage actually hurt validation performance. That is, the stronger regularization may have caused the model to underfit.

Part B

The active predictor counts for ridge at $C = 0.01$ is 561, whereas the active predictor counts for lasso at $C = 0.01$ is 60. One zero class-specific coefficient does not neccesarily remove a predictor from the whole multinomial model because the predictor has other coefficients that correspond to the remaining classes. Conceptually, it may be the case that a certain predictor does not play a role in identifying a certain class, but that does not neccesarily mean it is not important for identifying other classes.

An all-size-zero coefficient does not prove that the sensor feature is intrinsically irrelevant. Multinomial models are linear in their predictors, meaning that if the feature actually has a non-linear relationship with the activites, the model often will not detect this. Likewise, we are imposing a coefficient budget constraint, which means that predictors may be related to the outcome but not strong enough to warrant coefficient budget allocation. With such a large predictor space, it is also possible that many predictors are correlated or almost multicolinear; conceptually, this means that predictors are tracking almost the same underlying signal. In these circumstances, lasso generally selects one predictor from the group to keep, and this decision usually fringes on how well aligned the random noise component of the predictor is with the signal. Since the noise component of the predictors will vary from sample to sample, another sample may drop the coefficient it kept here and keep the coefficient it dropped here. In cases of near colinearity or correlation, it is therefore more appropriate to understand these related predictors as essentially capturing the same underlying signal, meaning that if any of these coefficients are non-zero, then this underlying signal is important for predicting the activities.

Part C

Based on the validation log loss, the configuration selected was the OvR ridge model ($C = 1$), which had the best validation performance. After refitting on all of the training data, this configuration's test accuracy was 0.9579, and its test log loss was 0.1545.

Test results should not be used to switch configurations. The point of test data is that it does not influence model fit or selection in any capacity. By enforcing this separation, we ensure that our test metrics are unbiased estimates of prediction performance on unseen data, and these numbers in turn provide insight on model generalization. Changing configurations based on test performance comprises this relationship, and the reported metrics will overestimate the ability of our model to generalize.

The setting considered here does not capture all possible deployment conditions. For one, the age range of subjects is 19-48, meaning the elderly and children are omitted. Acitvities for these demographics may look very different due to increased/decreased mobility and physical size. Additionally, this model only captures six types of activities, and these activities do not adequately capture the human range (biking, for instance).

Lastly, participants were instructed to keep their phone on their waist, which may not be typical for most people.